HƯỚNG DẪN KIỂM TRA & NGHIỆM THU ĐỒ ÁN BTL

HỌC PHẦN: KỸ THUẬT LẬP TRÌNH AN TOÀN (CSE703093 / CSE703153)

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng — Đại học Phenikaa

Nhóm 11: Chủ đề 4 — Dự án Fuzzing (FZP1 Binary Parser)

. . . . . .

1. TỔNG QUAN DỰ ÁN

- **Đề tài:** Phân tích Cú pháp Gói tin Nhị phân FZP1 & Đo lường Đối sánh 3 Trường phái Fuzzing (Black-box vs Greybox vs White-box Concolic SMT Z3).
- **Thành viên nhóm:** Lưu Đức Hiệp (50%) & Hà Nguyễn Trúc Linh (50%) — *[Mã sinh viên và Lớp bảo mật theo quy định]*.
- **Cấu trúc gói nộp:**
- `specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`: Tệp đặc tả 10 sheet chuẩn SSOT theo yêu cầu môn học.
- `source/`: Mã nguồn C bản lỗi (`fzp1_parser_v0.c`) và bản vá CERT C (`fzp1_parser_v1.c`).
- `report/BaoCao_BTL_Nhom11.docx` (và bản sao tại thư mục gốc): Báo cáo Word chính thức chuẩn định dạng đồ án Phenikaa.
- `report/BaoCao_BTL_Nhom11.pdf`: Bản PDF tương ứng lưu trữ theo chuẩn môn học.
- `results/`: Dữ liệu thô JSON của 30 lượt chạy benchmark, biểu đồ và nhật ký kiểm định.

. . . . . .

2. CÁC LỆNH KIỂM TRA VÀ TÁI LẬP KẾT QUẢ (REPRODUCIBILITY)

Bước 1: Biên dịch mã nguồn C

Môi trường Linux / WSL / Clang (AddressSanitizer):

```
gcc -fsanitize=address -O0 -g source/fzp1_parser_v0.c -o source/fzp1_parser_v0_asan
gcc -fsanitize=address -O0 -g source/fzp1_parser_v1.c -o source/fzp1_parser_v1_asan
```

Môi trường Windows (MinGW-w64 UCRT64 với Stack Canary):

```
gcc -Wall -Wextra -O0 -g -fstack-protector-all source/fzp1_parser_v0.c -o
source/fzp1_parser_v0.exe
gcc -Wall -Wextra -O0 -g -fstack-protector-all source/fzp1_parser_v1.c -o
source/fzp1_parser_v1.exe
```

Bước 2: Chạy kiểm thử tự động toàn diện (Pytest)

```
pytest tests/test_fzp1_parser.py -v
# Kết quả mong đợi: 10 passed trong < 1.0 giây
```

Bước 3: Kiểm chứng lỗi và bản vá bằng tệp tấn công

```
# 1. Chạy bản lỗi v0 với gói tin payload > 16 byte (whitebox_s0_crash_0.bin)
# - Trên Linux/Clang ASan: Bắt lỗi Stack Buffer Overflow với shadow byte [f3] (xem
results/logs/log_asan_crash_v0.txt)
# - Trên Windows: SẬP TIẾN TRÌNH (STATUS_STACK_BUFFER_OVERRUN, Exit Code: -1073741795 /
3221225501)
.\source\fzp1_parser_v0.exe .\crashes\whitebox_s0_crash_0.bin

# 2. Chạy bản vá v1 với cùng tệp tấn công -> Quan sát cơ chế TỪ CHỐI AN TOÀN (CERT STR31-
C)
```

```
.\source\fzp1_parser_v1.exe .\crashes\whitebox_s0_crash_0.bin
# Thông báo: [SECURITY_REJECT] Payload length 25 exceeds maximum allowed buffer 16
# Mã lỗi trả về: 2 (Từ chối an toàn, 0 lỗi sập bộ nhớ)
```

Bước 4: Chạy lại Benchmark 30 lượt thử nghiệm

```
python scripts/run_fzp1_benchmark.py
python scripts/aggregate_results.py
python scripts/generate_charts.py
# Dữ liệu được cập nhật tại results/data_raw/ và biểu đồ tại results/figures/
```

Bước 5: Chạy công cụ thẩm định tính toàn vẹn dự án

```
python scripts/verify_project.py
# Kết quả mong đợi: 10/10 TIÊU CHÍ ĐẠT CHUẨN (PASS)
```

. . . . . .

3. TÓM TẮT KẾT QUẢ ĐẠT ĐƯỢC

- **Tỷ lệ tìm thấy lỗi qua 30 trials:**
- Black-box (Ngẫu nhiên mù quáng): **0% (0/30)** — Rào cản toán học P = 1/2^{40} ≈ 9.09 × 10^{-13}.
- Greybox (Biết Magic header): **80% (24/30)** — Trung vị 3.27 giây.
- White-box (Z3 SMT Solver): **100% (30/30)** — Thời gian giải Z3 trung bình **3.92 ms** (trung vị time-to-crash 0.22 giây), tìm ra crash ngay trong vòng lặp đầu tiên.
- **Chuẩn hóa bản vá:** Tuân thủ 100% **CERT STR31-C**, **CERT MEM31-C**, **CERT MEM30-C**.